

AMAL - TP 2

Complete hands-on with pyTorch

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowski - Laure Soulier

2025-2026

In this practical session, you will exploit the full power of pyTorch for defining, training and evaluating neural networks. We will particularly address :

- using PyTorch's optimizer
- defining networks using modules
- managing training data
- using GPU (**warning** : this will only work if you have a GPU card)
- using checkpointing (resuming training after a voluntary or involuntary interruption)

We will therefore use all the possibilities offered by PyTorch from this practical session onwards, and this framework will remain the same until the end of this module.

Prerequisites You must have already implemented a gradient descent loop for optimizing linear regression as seen in the previous practical session using autograd. It is not necessary to have done the manual implementation of functions and back-propagation.

1 Optimizer

Pytorch includes a very useful class for gradient descent, `torch.optim`, which allows :

- saving a few lines of code
- automating parameter updates
- abstracting the type of gradient descent used (SGD, Adam, rmsprop, ...)

A list of parameters to optimize is passed to the optimizer during initialization. The `zero_grad()` method allows resetting the gradient to zero and the `step()` method allows updating the parameters. The example below updates the parameters once every 100 iterations (mini-batch of 100).

```
w = torch.nn.Parameter(torch.randn(1,10))
b = torch.nn.Parameter(torch.randn(1))
## we optimize according to w and b, lr: learning rate
optim = torch.optim.SGD(params=[w,b],lr=EPS)
optim.zero_grad()
```

```

# Gradient reinitialization
for i in range(NB_EPOCH):
    loss = MSE(f(x,w,b),y) #Cost calculation
    loss.backward()         # Backpropagation
    if i % 100 == 0:
        optim.step()        # Update parameters w and b
        optim.zero_grad()    # Gradient reinitialization

```

The `Parameter` class is a wrapper of the `Tensor` class which allows among other things to automatically specify that the gradient is required for this tensor and also to mark this tensor as a parameter to optimize. This differentiation is essentially used in the `Module` class that we will see below to automatically recognize parameters from other inputs/constants. Its use is optional in the previous code snippet, it also works with a classic tensor (and the `requires_grad` flag set to true).

Question 1

Modify your code to include the optimizer rather than doing the updates manually. Compare your version, the one with the SGD optimizer and a final version with the Adam optimizer.

2 Module

In the PyTorch framework (and in most similar frameworks), the module is the basic building block for constructing a neural network. It allows representing in particular :

- a network layer (linear : `torch.nn.Linear`, convolution : `torch.nn.convXd`, ...)
- an activation function (tanh : `torch.nn.Tanh`, sigmoid : `torch.nn.Sigmoid`, ReLu : `torch.nn.ReLU`, ...)
- a cost function (MSE : `torch.nn.MSELoss`, L1 : `torch.nn.L1Loss`, CrossEntropy : `torch.nn.CrossEntropyLoss`, ...)
- but also regularization tools (BatchNorm : `torch.nn.BatchNorm1d`, Dropout : `torch.nn.Dropout`, ...)
- a set of modules ; in computer science terms, a module is an abstract container that can contain other containers : several modules can be put together to form a new more complex module.

The operation is very close to functions : a module actually encapsulates a function inherited from `torch.nn.Function` but in a way that automatically manages the parameters to learn. The `Parameter` class is used to create a parameter of the module. The parameter thus created is automatically added to the list of module parameters. The list of parameters is then accessible through the `parameters` method.

Just like a function, a module class is equipped with :

- a **forward** method that allows calculating the module's output from the inputs
- a **backward** method that allows performing backpropagation (locally).

In the case where the **forward** method only performs calculations using functions available in PyTorch, the **backward** method does not need to be implemented! Indeed, backpropagation can be handled by automatic differentiation. The constructor allows initializing the different components of the module.

Below is an example of a module class; it is essential not to forget to inherit it from the abstract class `nn.Module` (necessary in particular to properly register the parameters of sub-modules in the created module).

```
class MyLinearModule(torch.nn.Module):
    def __init__(self):
        super(MyLinearModule, self).__init__()
        # two linear sub-modules
        self.one = torch.nn.Linear(10, 100)
        self.two = torch.nn.Linear(100, 1)
    def forward(self, x):
        # define the forward behavior of the module
        return self.two(self.one(x))

m = MyLinearModule()
x = torch.randn(10, 10)
# The module's parameters (to pass to the optimizer)
print(list(m.parameters()))
# forward calculation either by explicit function call,
# or by module call (shorter)
y = m.forward(x)
y = m(x)
```

Question 2

Use the modules `torch.nn.Linear`, `torch.nn.Tanh` and `torch.nn.MSELoss` to implement a two-layer network : $\text{linear} \rightarrow \text{tanh} \rightarrow \text{linear} \rightarrow \text{MSE}$. Implement the gradient descent loop with the optimizer.

Now use a container - for example the `torch.nn.Sequential` module - to implement the same network. Browse the documentation to understand the difference between the different types of containers. What happens to the parameters of modules put together this way?

Finally use a class to implement the same network.

3 Managing data with Dataset and Dataloader

The `Dataset` and `Dataloader` classes make it easier to manage data in PyTorch. The `Dataset` class is an abstract class that allows specifying how an example is loaded, pre-processed, transformed, etc. and provides access through an iterator to each example of a dataset. The `Dataloader` class encapsulates a dataset and allows querying this dataset in various ways : specifying the mini-batch size, whether the order should be random or not, how examples are concatenated, etc.

To implement a `Dataset`, it is sufficient to define two methods `__getitem__(self, index)` and `__len__(self)` :

```
from torch.utils.data import Dataset, DataLoader
class MyDataset(Dataset):
    def __init__(self, ...):
        pass
    def __getitem__(self, index):
        """ returns a (example, label) pair corresponding to the index """
        pass
    def __len__(self):
        """ returns the size of the dataset """
        pass
```

One of the main advantages (apart from possible pre-processing) is that it is not necessary to load the entire dataset into memory, loading can be done on-the-fly when the i -th example is requested. Moreover, the pre-existing `TensorDataset` class allows building a dataset for a list of tensors passed as argument (the i -th example is an n-tuple composed of the i -th row of each tensor).

Once a dataset `MyDataset` is implemented, it is sufficient to create a `DataLoader` in the following way for example :

```
# Creating the dataloader, specifying batch size and random order
data = DataLoader(MyDataset(...), shuffle=True, batch_size=BATCH_SIZE)
for x,y in data:
    #x,y is a batch of size BATCH_SIZE
```

Question 3

Implement a dataset for the MNIST dataset that returns an image as a normalized vector between 0 and 1, and the associated label (without using `TensorDataset`). Test your dataset with a dataloader for different batch sizes and explore the dataloader options.

You can refer to the official documentation.

To load MNIST :

```
from sklearn.datasets import fetch_openml
#set DATA_PATH to the directory where data will be downloaded.
```

```
x,y = fetch_openml('mnist_784',return_X_y=True,as_frame=False,data_home=DATA_PATH)
```

4 GPU

To take advantage of a GPU's computing power, you must explicitly tell **PyTorch** to load tensors on the GPU as well as the module (i.e., the module's parameters). It is not possible to perform operations when some tensors are on GPU and others on CPU (an error message is displayed in this case). The `to(device)` operator for tensors and modules allows copying them to the specified GPU (or CPU) (note that the operation is not *in-place* for tensors - for modules it is - you must save the result in a variable that will be on the right device). Below is an example :

```
#allows selecting the gpu if available
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = ...
model = model.to(device) #loading the module (parameters) on device
x = x.to(device) # load data on device
y = model(x) # gpu computation
y = y.to('cpu') # if we want to put back on cpu
```

5 Checkpointing

It is often useful to save the model as training progresses, for example to be able to resume calculations in case of interruption. **PyTorch** has a very practical mechanism for this through the `state_dict()` function which returns the important parameters of a model (the learning parameters) in dictionary form. But it is also often necessary to know the state of the optimizer used to resume calculations. This same `state_dict()` function also allows saving the values of important parameters for the optimizer used. In practice, the high-level functions `torch.save()` and `torch.load()` make it very easy to save and load the desired information and additional information : they will use Python's usual serializer `pickle` for standard structures and the `state_dict()` functions for **PyTorch** objects.

An example of use below :

```
from pathlib import Path
savepath = Path("model.pch")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
class State:
    def __init__(self,model,optim):
        self.model = model
        self.optim = optim
        self.epoch, self.iteration = 0,0
if savepath.is_file(): # if the file already exists
    with savepath.open("rb") as fp:
        state = torch.load(fp) #we resume from the saved model
else:
```

```

model = ...
model = model.to(device)
optim = ...
state = State(model, optim)

for epoch in range(state.epoch, ITERATIONS):
    for x,y in train_loader:
        state.optim.zero_grad()
        x = x.to(device)
        xhat = state.model(x)
        l = loss(xhat, x)
        l.backward()
        state.optim.step()
        state.iteration += 1
    with savepath.open("wb") as fp:
        state.epoch = epoch + 1
        torch.save(state, fp)

```

6 Implementing an autoencoder

An autoencoder is a neural network that allows projecting (*encoding*) a dataset into a very low dimensional space (it *compresses* the dataset). The output dimension is the same as the input, it is trained so that the output is as close as possible to the input - $f(\mathbf{x}) \approx \mathbf{x}$ - with a least squares cost for example or a cross entropy cost. We call *decoding* the calculation of the output from the projected data.

Question 4

Implement an autoencoder class (inherited from Module) according to the following architecture : $linear \rightarrow Relu$ for the encoding part and $linear \rightarrow sigmoid$ for the decoding part. The decoder weights usually correspond to the transpose of the encoder weights (what is the advantage?).

Question 5

Conduct an experimental campaign for the autoencoder on MNIST and also for architectures with 2 or 3 hidden layers using all the tools presented (GPU, checkpointing, **tensorboard** in particular).

For **tensorboard**, it is possible not only to plot scalars as a function of iterations, but also latent states (**add_embedding** method), precision/recall curves (**add_pr_curve** method)

or images (`add_image` method).

You can also visualize the continuity of the obtained **embeddings** (the representations in the latent space) : take two images $\mathbf{x}^1$ and $\mathbf{x}^2$ of different classes, calculate their representation $\mathbf{z}^1$ and $\mathbf{z}^2$, then display the images corresponding to the decoding of the interpolation of their representation $\lambda * \mathbf{z}^1 + (1 - \lambda)\mathbf{z}^2$ for some values of λ between 0 and 1.

Question 6

Implement the Highway network. Compare the performance against other classical architectures.